TURING CLUB OF PROGRAMMERS
2026-27

CODE UTSAVA X.0

Code.Innovate.Celebrate

Central India’s Largest Coding Fest

Problem Statement 1:

Domain:

@Computer Vision, @Industrial Automation, @AI/ML, @Quality Assurance, @Smart Manufacturing

Background:

Insteel plants,manufacturing facilities, and heavy metallurgical industries, billet production requires strict dimensional accuracy and precise trace identification (e.g., heat number, batch code, length, width, and surface quality). Currently, many production lines rely on manual physical measurements or operator visual checks to record billet details.

Manual inspections are slow, prone to human error, hazardous in high-temperature environments, and create operational bottlenecks. Furthermore, manually entering inspection data into tracking systems leads to missing, incomplete, or corrupted records, compromising quality control and end-to-end traceability across the supply chain.

Problem Statement:

There isacritical need for an automated, non-contact computer vision system that captures realtime video or camera feeds of billets on a conveyor or cooling bed, measures their dimensions, extracts identifying details, and logs all inspection records into an Excel/CSV spreadsheet for tracking and audit purposes.

The system must:

- Capture live camera frames of passing billets under varying industrial lighting conditions.
- Measure physical dimensions (such as length, width, height/diameter) and detect structural defects or cross-sectional variations.
- Read stamped, painted, or laser-etched markings (OCR for heat numbers, batch IDs, or barcodes/QR codes).
- Process measurements automatically and append all extracted metrics (timestamp, billet ID, dimensions, pass/fail status) into an Excel workbook or database-exportable tracking log.

- Alert operators instantly if dimensions fall outside specified manufacturing tolerancelimits.

Deliverables:

1. CameraFeed & Vision Processing Engine: A computer vision pipeline (usingOpenCV, PyTorch, or TensorFlow) capable of real-time object detection,segmentation, and dimensional measurement calibration.
2. OCR&Detail Extraction Module: Text identification/OCR engine trained to readstamped, etched, or painted identifiers on hot/cold steel billet surfaces.
3. AutomatedExcel Data Logging System: An automated reporting integration(e.g., via pandas/openpyxl) that logs timestamped measurements, status tags (Pass/Fail/Rework), and extracted identifiers into structured .xlsx or .csv files.
4. Operator Dashboard / Alert Interface: A graphical interface showing the live camera feed, realtime bounding boxes/measurements, and instant alert notifications for out-of-tolerance billets.

Impact:

- **Increased Operational Safety:** Eliminates the need for manual physical checks near hot billets or moving heavy machinery.
- **100% Traceability & Compliance:** Replaces manual record-keeping with automated, error-free Excel tracking logs accessible for audits.
- **Higher Production Throughput:** Real-time camera scanning prevents bottlenecking on conveyor lines.
- **Minimized Scrap & Cost Reduction:** Early detection of out-of-tolerance billets prevents defective batches from reaching downstream operations.

Performance & Technical Requirements:

- Measurement Accuracy: Dimensional error margin within <= +-1% of actual physical dimensions after camera calibration.

- Processing Speed: Real-time processing speed (≥15−30 FPS) or inspection latency under 2 seconds per billet.

- OCR Reliability: Minimum 90% accuracy on reading stamped or painted billet identification text.

- Robustness: Resilient to challenging factory conditions such as variable lighting, dust, and mild surface scale.

- Data Integration: Seamless append-mode logging into Microsoft Excel without file lock or corruption issues during continuous scanning

Problem Statement 2:

Domain:

Electrical Power Systems, Smart Grids, IoT, Machine Learning, Anomaly Detection, Time-Series Analysis, Energy Analytics and Generative AI

Background:

Electricity distribution networks face losses due to technical losses, faulty/tampered meters, communication failures, abnormal consumption and electricity theft. Such losses may occur at the consumer-meter level as well as between distribution points such as transformers, feeders and towers/poles. Traditional threshold-based and manual methods can produce false alarms because consumption changes may also occur due to seasonal variations, consumer behaviour or meter faults.

The proposed system is a software-centric intelligent platform that analyses historical and continuously arriving electricity data using machine learning, time-series analysis, consumer profiling and energy accounting. It generates anomaly scores, probable causes, supporting evidence and inspection priorities. IoT is an additional real-time layer; if physical hardware is unavailable, a continuous virtual smart-meter simulator will generate and stream realistic readings.

Problem Statement:

Develop a system that can:

- Detect abnormal consumption, sudden load changes, irregular and missing readings.
- Distinguish probable theft/tampering from meter faults, communication failures and legitimate variations.
- Analyse temporal, historical and behavioural patterns.
- Build consumer baselines and compare similar consumers.
- Combine consumption, load, meter-status and historical signals.
- Generate explainable anomaly/risk scores with supporting evidence.
- Prioritise consumers for field inspection.
- Identify suspicious consumer clusters at transformer/feeder level.
- Process continuously arriving data through virtual or physical IoT sources.

Prototype Scope & Inputs:

The prototype will focus on a feeder, distribution transformer or group of consumers, with primary emphasis on the software intelligence layer.

Inputs:

- Historical consumption and meter readings.
- Consumer ID, category, sanctioned load and tariff.
- Meter status and communication information.
- Transformer/feeder energy data.
- Theft/anomaly records.
- Optional weather/seasonal information.
- Continuous virtual smart-meter readings.

Dataset Sources: Kaggle Electricity Theft Detection Dataset and Electricity Theft Data. These are initial sources; preprocessing, feature engineering and assumptions should be documented.

The system should demonstrate **normal consumption, theft/tampering, meter malfunction,communication failure and legitimate abnormal consumption.**

Illustrative Case:

If a consumer normally uses 350–400 kWh/month but suddenly drops to 80–100 kWh for three months, the system compares the readings with historical behaviour, similar consumers, load characteristics and meter status.
If communication is normal and the pattern persists, the system may classify it as probable theft/tampering with a high anomaly score. If communication failed during the same period, it should instead identify a communication/data issue.
The output should contain risk level, probable cause, confidence, supporting evidence and recommended inspection.

Dataset Sources

The prototype can use the following publicly available Kaggle datasets:

- Electricity Theft Detection Dataset — Kaggle Dataset
- Electricity Theft Data — Kaggle Dataset

The dataset sources, preprocessing steps and assumptions should be documented.

The system should demonstrate:
1. Electricity theft/tampering.
2. Meter malfunction.
3. Communication failure.
4. Legitimate abnormal consumption.
5. Normal consumption.

Deliverables:

1. Advanced Anomaly Detection & Consumer Profiling Engine
Build the core software engine for data cleaning, consumer baselines, time-series/behavioural feature extraction, anomaly detection, risk scoring and handling missing/noisy/irregular readings.
Possible approaches: LightGBM/XGBoost, Isolation Forest, LSTM Autoencoder, statistical methods or a hybrid ML pipeline. Multiple signals should be combined instead of relying on a single threshold.

2. Theft Risk & Cause Classification
For each consumer, generate:

- Anomaly score and risk level.
- Probable cause and confidence.
- Key contributing factors and detection period.
- Recommended action.

Possible causes include theft/tampering, meter malfunction, communication failure, abnormal
consumption, seasonal variation and normal behaviour. Cases with insufficient evidence should
be marked Requires Review.

3. Explainable Investigation Report
Generate reports containing consumer/meter details, historical vs current consumption, deviation,
anomaly score, time period, behavioural pattern, supporting features, probable cause, confidence,
recommended inspection and priority.
The system must explain **why a consumer was flagged**, not only provide a prediction.

4. Reviewer Dashboard

Develop a web dashboard to:

- Filter consumers by risk, feeder, transformer and category.
- View historical and real-time consumption.
- Compare actual vs baseline consumption.
- Inspect anomaly explanations.
- View suspicious clusters and transformer/feeder losses.
- Update case status.
- Export CSV/JSON/PDF reports.

5. Transformer/Feeder Analysis

Analyse transformer/feeder energy data to identify unusually high unexplained losses and correlate
them with consumer-level anomaly scores. This helps identify suspicious areas and clusters.

6. Continuous Virtual IoT Monitoring — Additional Deliverable

IoT will be an additional real-time layer, not the primary dependency.
If hardware is unavailable, build a Virtual Smart Meter Simulator that continuously generates
readings for multiple consumers, including:
Voltage, current, power and energy.

- Normal consumption patterns.
- Theft/tampering patterns.
- Meter faults.
- Communication failures.
- Realistic noise and variations.

The readings should continuously stream through MQTT/WebSocket/API → Backend → ML Engine
→ Database → Dashboard/Alerts. The dashboard should update as new readings arrive.
Optional hardware: ESP32 + current/voltage/energy sensor can replace the simulator without
changing the main software architecture.

7. Regression & Scenario Test Suite
Test the system using normal consumption, theft/tampering, meter malfunction, communication
failure, seasonal variation, temporary legitimate reduction, new consumers, transformer discrepancies
and missing/noisy readings.

8. Reproducible Demo
Provide source code, datasets, preprocessing pipeline, trained models, backend/API, database,
dashboard, virtual IoT simulator, tests, sample reports, setup instructions, assumptions, limitations
and evaluation results.
Demo:
Historical Data → Consumer Profiling → ML Detection → Risk/Cause Classification →
Explanation → Virtual IoT Stream → Real-Time Anomaly → Alert → Inspection Priority

Performance & Technical Requirements:

Evaluate the system using Precision, Recall, F1-Score, ROC-AUC/PR-AUC, false-positive/false-negative rates, detection latency and processing performance.

The system should handle missing/noisy/duplicate readings, irregular sampling, communication failures, seasonal variations, different consumer categories and limited history.
For continuous IoT data, measure number of consumers, reading frequency, throughput, processing latency, detection latency and dashboard update latency.

Problem Statement 3:

Domain:
@Cybersecurity, @Agentic AI, @Generative AI, @RAG Security, @NLP

Background:
AIagents increasingly read documents, browse webpages, and call tools (files, email, search, databases) to complete tasks on a user's behalf. Retrieved content is fed into the model's context alongside the user's request, and the model cannot reliably tell a genuine user instruction from data it has retrieved.

This creates a serious and largely unsolved risk called prompt injection. An attacker hides instructions inside a document, webpage, or tool response. These instructions can be plainly written, encoded (base64, hex, zero-width characters), disguised as a fake "system message", or embedded in a tool's output. When the agent reads this content, it may be hijacked into leaking confidential files, modifying records, or sending emails the user never authorized.

Existing defences are usually single-layer: a keyword filter or a prompt-level warning. These are easily bypassed, and once an injection succeeds there is nothing to stop the harmful action. Organisations deploying agents in procurement, finance, HR, and customer support need a protection layer that checks both what the agent reads and what the agent does.

Problem Statement:
There is a critical need for a software protection layer that sits around a tool-using RAG agent, detects hidden malicious instructions in untrusted content, and verifies that every action the agent proposes stays within the scope of what the user actually asked for.

For example, an agent asked to compare vendor quotations must complete the comparison even when one quotation contains a hidden instruction to email a confidential file to an external address. The attack should be stopped, and the legitimate task should finish.

The system must:

- Extract the permitted tools, resources, and recipients from the user's request to form an authorised scope for the task.
- Clearly separate trusted user instructions from untrusted content retrieved from documents, webpages, and tool responses.
- Scan untrusted content for hidden, encoded, or impersonated instructions before it reaches the agent's context, and flag, sanitise, or quarantine it.
- Inspect every proposed tool call and detect attempts to access, modify, or disclose information beyond the permitted scope.
- Allow legitimate actions, block unauthorised ones, and request human confirmation when permission is ambiguous.
- Record the evidence and reasoning behind every decision in an audit log.
- Keep legitimate work running with minimal over-blocking and low added latency.

Deliverables:

1. **Sandbox Environment & Vulnerable Baseline Agent:** A LangChain/LangGraph tool-using RAG agent over a vendor-quotation corpus with mock tools (read_file, search_web, send_email, write_record) and a fake confidential folder. The baseline must be genuinely exploitable, so that attacks demonstrably succeed when protection is off.

2. **Content Firewall (Input-Side Defence):** A layered detection module combining regex and heuristics, a decoding pass (base64, hex, unicode, zero-width characters, HTML comments), an LLM-based instruction classifier, and spotlighting/delimiting of untrusted content as data only.

3. **Action Guard (Output-Side Defence):** A module consisting of an LLM-based scope extractor, rule-based checks (tool scope, resource paths, approved recipients, confidential-to-external data flow), and a provenance/taint check. It returns an Allow / Block / Ask Human decision for each proposed tool call.

4. **Audit Log & Evaluation Harness:** A structured JSON audit log recording the triggering content snippet, the rule violated, and the decision. The

harness runs an attack suite of 25–30 attacks across 5 categories (plain, encoded, fake system message, tool-response injection, multi-step exfiltration), split into development and unseen sets, plus 15–20 benign tasks.

5. **Demo Interface:** A Streamlit dashboard (or clean CLI) showing unprotected and protected agents side by side on the same attack, an attack selector, a live audit log, human-confirmation prompts, and a results table.

Impact:

- **Data Protection:** Prevents leakage of confidential files, credentials, and records through hijacked agents.
- **Defence in Depth:** Even if an injection slips past the content firewall, the action guard still stops the harmful action.
- **Trust & Compliance:** Every allow, block, and confirmation is logged with evidence, giving auditable proof of agent behaviour.
- **Safe Adoption of AI Agents:** Lets organisations give agents real tool access without relying on the model alone to resist manipulation.
- **Continuity of Work:** Legitimate tasks complete as normal, with no blanket shutdown of agent capabilities.

Performance & TechnicalRequirements:

- **Baseline Vulnerability:** The unprotected agent must be hijacked by at least 70% of the attack suite, showing the threat is real.
- **Attack Catch Rate:** The protected system should block at least 85% of development attacks and report an honest, separately measured rate on unseen attacks. Misses must be reported and analysed.
- **Task Completion:** At least 90% of benign tasks, and poisoned-but-legitimate tasks such as the quotation comparison, must still complete correctly.
- **False-Positive Rate:** No more than 10% of benign actions unnecessarily blocked or escalated.
- **Latency Overhead:** Added latency under 2 seconds per request, with a per-layer breakdown (firewall, scope extraction, guard) reported.
- **Explainability:** Every intervention must have a human-readable reason and the evidence that triggered it.

- **Robustness:** The defence should generalise to unseen attack phrasings and encodings, not only those used during development.
- **Safety of Testing:** All tools operate on mock data, with no real emails sent and no real files touched.

Problem Statement 4:

Domain:

@DatabaseOptimization, @ArtificialIntelligence, @DataPrivacy, @ReinforcementLearning, @GraphNeuralNetworks

Background:

Database administrators struggle to maintain optimal performance in massive relational databases as application query patterns evolve over time. Identifying missing composite indexes, implementing sharding, or manually rewriting complex joins is a heavy operational burden that risks disrupting live production environments. Furthermore, leveraging AI to solve this introduces severe security risks, as exposing raw, sensitive company data to AI tuning models violates privacy and compliance standards.

Problem Statement:

There is a critical need for an AI-driven interactive dashboard that automates database performance tuning while enforcing strict data guardrails. The system must never read raw production data; instead, it must abstract incoming data patterns to feed safe, anonymized representations to the AI.

For example:
For a query such as “Why is the weekly sales reporting dashboard timing out?”, the dashboard should analyze the database's recent execution plans using only metadata and masked attributes. It identifies a highly nested join causing a sequential scan on a massive, unpartitioned table. The system then safely rewrites the SQL, simulates the changes, and presents an evidence-backed response like: “Partitioning the target table by month, rewriting the join, and adding a composite index on (region_id, transaction_date) will reduce query time by 85%, with only a 2ms simulated increase in write latency. Zero raw data was exposed during this analysis.”

The system must:

- Capture live camera frames of passing billets under varying industrial
  Abstract incoming data patterns using attribute bitmasking and structural metadata hashing to guarantee anonymity.
- Ingest these anonymized slow query logs and historical execution plans securely.
- Recommend optimal composite indexes and robust table partitioning or sharding strategies based on abstracted access patterns using Reinforcement Learning.

- Automatically rewrite inefficient SQL queries. Use AI and Graph Neural Networks (GNN) to provide natural language explanations of complex query execution trees.
- Leverage database extensions (e.g., PostgreSQL Hooks) to simulate the exact impact of proposed structural changes on write-latency and storage without applying them to production.

Deliverables:

- **Data Obfuscation & Ingestion Engine:** A secure middleware module utilizingData Obfuscation/Bitmasking to convert slow query logs and executionplans into structural metadata hashes before AI ingestion.
- **AI & RL Optimization Agent:** A Reinforcement Learning engine that analyzesabstracted access patterns to recommend optimal composite indexes,Database Sharding/Partitioning schemes, and automatically rewritesinefficient SQL queries.
- **Execution Tree Analytics Module:** A Graph Neural Network (GNN) integratedwith AI to map complex query execution trees and translate performance bottlenecks into clear, natural language explanations.
- **Simulation Sandbox:** A robust simulation framework leveraging PostgreSQL (Hooks/Extensions) to calculate the theoretical impact of structural recommendations (e.g., storage overhead, write-latency spikes) purely in a sandbox state.
- **Interactive Dashboard:** A user-facing AI Agent interface that allows administrators to query performance issues naturally, view GNN-backed explanations, and assess simulated metrics prior to authorizing production deployment.

Impact:

- **Data Protection & Compliance:** Ensures zero exposure of raw, sensitive company data to AI models, strictly adhering to enterprise privacy standards.
- **Operational Efficiency:** Drastically reduces the time database administrators spend manually analyzing slow queries, rewriting SQL, and planning shards.

- **Zero-Risk Testing:** Simulates structural alterations safely, preventing disastrous performance degradations in live production environments.
- **Explainability & Trust:** Demystifies black-box AI recommendations by visually and textually explaining the underlying execution paths and bottlenecks.

Performance & Technical Requirements:

- **Zero Data Exposure:** 100% of sensitive raw data must be successfully bitmasked or hashed prior to AI processing.
- **Simulation Accuracy:** The simulated impacts on write-latency and storage via PostgreSQL Hooks must closely mirror actual theoretical metrics without applying physical changes to live tables.
- **Optimization Efficacy:** The system's rewritten SQL and indexing recommendations should demonstrably reduce query execution time for targeted workloads (e.g., targeting a >50% improvement on inefficient joins).
- **Explainability:** Every optimization intervention must include a human-readable GNN-generated reason and the specific metadata evidence that triggered the recommendation.
- **Robustness:** The RL model must generalize effectively to unseen, evolving query patterns rather than only optimizing for static historical logs.

Problem Statement 5:

Domain:

@FinancialIntelligence, @Macroeconomics, @AgenticAI, @MultiAgentSystems, @PredictiveAnalytics, @PortfolioManagement

Background:

Financial analysts and portfolio managers struggle to synthesize the overwhelming volume of real-time market news, macroeconomic indicators, and alternative data like extreme weather forecasts into actionable investment strategies. Manually aggregating these fragmented, unstructured data streams to predict how a sudden, localized event will impact specific asset classes or supply chain-dependent equities is a slow, error-prone process that leads to missed trading opportunities and unmitigated portfolio risks.

Problem Statement:

There is acritical need for an AI-powered financial intelligence terminal that continuously ingests global news feeds, real-time weather, and historical market data to forecast equity and commodity impacts using an autonomous multi-agent portfolio management system.

For example:
For a query such as “How will the forecasted Category 4 hurricane in the Gulf of Mexico affect our current energy holdings?”, the system should aggregate live weather tracking data, retrieve historical market reactions to similar storms, and analyze real-time news sentiment. It should then generate an evidence-backed response like: "The hurricane is projected to disrupt 40% of Gulf refining capacity for 5 days. Recommendation: Hedge long positions in Gulf-based refiners and temporarily increase exposure to natural gas futures, as our combined sentiment and weather models forecast an 8% spike in short-term energy prices."

The system must:

- Ingest and index multi-modal data streams (real-time news feeds, macro indicators, weather patterns, and market price series) using vector databases and specialized financial APIs.

- Deploy a collaborative multi-agent portfolio management orchestration engine (e.g., via LangGraph/LangChain) with specialized agents for sentiment analysis, risk modeling, weather impact correlation, and execution strategies.
- Utilize specialized agents to query historical macroeconomic event parallels and cross-asset correlations from vector databases (Pinecone/Weaviate) to inform multi-agent portfolio management decisions.
- Perform automated sentiment analysis, macro trend modeling, and quantitative risk evaluation across asset classes.
- Generate automated hedging strategies, asset reallocation proposals, and quantitative risk assessments in response to natural language queries.
- Present actionable insights through a dynamic, real-time terminal dashboard (Streamlit or Next.js) with drilldown capability for full auditability of agent decisions.

Deliverables:

- **Multi-Modal Data Ingestion Pipeline:** An automated ingestion architecture connecting Financial APIs (Alpha Vantage/Polygon) and Weather APIs, backed by a high-throughput Vector Database (Pinecone/Weaviate) for historical event retrieval.
- **Multi-Agent Portfolio Management Engine:** A stateful agentic orchestration system (built with LangChain/LangGraph) comprising dedicated agents for Sentiment Analysis, Weather & Macro Impact Analytics, Quantitative Risk Modeling, and Automated Hedging Strategy Generation.
- **Quantitative Risk & Forecast Simulator:** A specialized analytical engine that computes market exposure metrics, estimates potential price impacts, and generates multi-asset hedging strategies based on crosscorrelated data streams. **Natural Language Query & Reasoning Engine:** An LLM-powered conversational interface capable of parsing complex financial queries, executing multi-step reasoning across agents, and grounding responses in real-time data evidence.
- **Interactive Intelligence Terminal:** A responsive dashboard (Streamlit/Next.js) displaying live multi-agent execution graphs, interactive

price/weather charts, risk exposure breakdowns, and explicit agent trade/hedge recommendations.

Impact:

- **Proactive Risk Mitigation:** Identifies localized alternative data risks (e.g., extreme weather, geopolitical events) and recommends automated hedges before market pricing fully adjusts.
- **Autonomous Multi-Agent Synergy:** Eliminates manual cross-silo analysis by orchestrating dedicated agents for market data, sentiment, and risk management into a unified multi-agent portfolio management workflow.
- **Rapid Actionable Insights:** Synthesizes unstructured news, weather, and price data into quantifiable forecasts and clear portfolio hedging strategies in seconds.
- **Grounded & Auditable Decisions:** Ensures all trade recommendations are backed by explicit multi-agent evidence logs and historical retrieval references.

Performance & Technical Requirements:

- **Ingestion & Retrieval Latency:** Real-time news and weather data streams must be processed, embedded, and indexed within sub-second intervals to support low-latency agent queries.
- **Multi-Agent Orchestration Accuracy:** The multi-agent workflow must correctly decompose complex natural language queries, delegate sub-tasks to specialized agents, and synthesize coherent, evidence-grounded multi-agent portfolio management recommendations.
- **Forecast Reliability:** Sentiment and multi-modal correlation models should achieve demonstrably higher predictive alignment on market shifts compared to single-data-source baselines.
- **Auditability & Explainability:** Every recommendation produced by the multi-agent system must include a step-by-step breakdown of data sources, sentiment scores, and historical event matches that informed the hedging decision.
- **System Robustness:** The terminal interface and multi-agent pipeline must gracefully handle API rate limits, missing data streams, and volatile real-time market updates without failing or hallucinating financial metrics.